

Plan Your System Design Preparation

System design is a structured discussion of requirements, data, interfaces, capacity, and failure. Start with a small correct design, identify the important bottlenecks, and deepen only the parts that matter. A defensible trade-off is more valuable than a diagram containing every fashionable component.

For: Engineers who understand individual services and want a repeatable method for architecture interviews without memorizing large diagrams or selecting technologies before requirements.

Suggested pace: 6 weeks, 6-8 hours per week | Backend and database basics required

What you will be able to do

- Clarify use cases, non-goals, and quality requirements before architecture.
- Define APIs, data models, and end-to-end read/write paths.
- Estimate capacity with explicit assumptions and consistent units.
- Explain consistency, retries, recovery, observability, and rollout trade-offs.

Build the right mental model

Requirements control the answer

Ask who uses the system, which operations matter, what scale is assumed, and what correctness or latency is required. A private messaging system and a public feed may share components but have different privacy, ordering, and fan-out constraints.

Start with a coherent baseline

One API layer and one durable database may be enough to explain the contract. Add caches, queues, partitions, and replicas when a stated requirement justifies them. Each component introduces its own failure and consistency behavior.

Follow data through failures

Walk a successful write, a read, a duplicate request, and a dependency failure. State the durable acknowledgement boundary and what is recoverable. A diagram without these paths does not show whether the system preserves its promises.

Study with evidence.

For every practice design, write a one-page contract before drawing. Use the same delivery structure, but choose deep dives from the problem. Keep a failure log of missed assumptions and revisit the design with one changed requirement.

Weeks 1-2: Contracts and foundations

Review APIs, SQL/NoSQL access patterns, indexes, HTTP, caching, replication, and queues. Design a small link or task service with explicit data ownership and errors. Estimate request rates from daily volume correctly.

Deliverable and check: Two small designs with APIs, schemas, assumptions, and complete read/write paths.

Week 3: State and correctness

Practice idempotency, transactions, consistency, leases, and authorization. Trace timeouts and partial failures. Explain which operations can tolerate stale data and which require an authoritative check.

Deliverable and check: A reservation or payment-like exercise with a named invariant and duplicate-request handling.

Weeks 4-5: Scale and operations

Study partitioning, hot keys, fan-out, backpressure, large objects, and regional failure. Add observability and capacity budgets. Deep-dive one role-relevant domain rather than learning many shallow product diagrams.

Deliverable and check: Two larger designs with bottlenecks, failure modes, recovery objectives, and operational signals.

Week 6: Mock and revise

Run timed design discussions with a peer who changes one constraint. Practice explaining rejected alternatives and uncertainty. Review actual role expectations and use the final days to strengthen recurring weak areas.

Deliverable and check: A mock scorecard showing clear prioritization, coherent data flow, and defensible trade-offs.

A repeatable practice session

Spend about 15 minutes recalling the previous topic without notes, 45 minutes on today's problem or lab, and 15 minutes explaining and checking the result. Record the failed assumption, not just the final answer. Revisit weak material after a few days and again the following week.

Apply it. Explain it.

Design a notification service from a contract

A product needs email and push notifications with user preferences, retries, and delivery status. The service must not block the original user request while waiting for an external provider.

A worked approach

1. Define notification ID, recipient, channel, template version, and preference snapshot or recheck policy. Specify accepted, queued, sent, failed, and cancelled states. Clarify whether a provider acceptance means delivery or only receipt by the provider.
2. Persist the request and an outbox event atomically, then have workers consume it from a durable queue. Use stable operation IDs, bounded retries, provider timeouts, and per-tenant fairness. Keep provider-specific details behind adapters.
3. Walk an ambiguous provider timeout, duplicate queue delivery, opt-out change, and provider outage. Explain deduplication limits, dead-letter handling, and the metrics for queue age, delivery attempts, errors, and suppressed sends.

Expected result

The architecture follows a specific requirement: asynchronous delivery with durable status and controlled retries. It does not claim exactly-once user-visible delivery when the external provider cannot support that boundary.

Verify the result

- Is the request durable before the API acknowledges acceptance?
- Can an opt-out be honored before a delayed send?
- Are retry load and backlog bounded during an outage?

Extend the exercise

Add a high-priority security-alert channel. Decide how priority affects fairness and reserved capacity without starving ordinary notifications or violating user and policy requirements.

Make the learning stick.

How much capacity math should I do?

Enough to influence design choices. Estimate request rates, data volume, bandwidth, and concurrency where they matter, then identify which assumption drives the result. Avoid lengthy arithmetic that does not change the architecture.

What if I do not know a technology's exact limit?

State an illustrative assumption, identify the measurement or documentation needed, and reason from the contract. Do not invent benchmark numbers or claim a product provides guarantees you have not verified.

What distinguishes a strong deep dive?

It follows a meaningful risk from requirement to mechanism, failure behavior, and validation. Explaining a hot-key mitigation or transaction boundary thoroughly is stronger than naming several unrelated services.

Avoid these traps

- Choosing a stack before clarifying the problem.
- Drawing components without read, write, and failure paths.
- Claiming scale or guarantees without explicit assumptions.

Ready for the next step?

- Requirements and non-goals are prioritized.
- APIs and data ownership are clear.
- Capacity and failure analysis influence decisions.
- Trade-offs remain coherent when one requirement changes.

Reliable next reads

- [Azure Architecture Center](#)
- [Google SRE book](#)
- [Hello Interview: system design introduction](#)

An independent preparation guide, not an employer-endorsed syllabus or a guarantee of an offer. Company examples are practice contexts, not verified past interview questions. Use only public, synthetic, or authorized material in exercises.